

CICIDS2017 — สรุปการเทรนทุกรอบ

การเทรนฝั่ง 2017 ทำโดยเพื่อนร่วมทีม ไฟล์ที่ส่งมอมมาพร้อมข้อมูล สคริปต์ และการยืนยัน seed อยู่ใน `cicids2017_handoff.zip`

ลำดับรอบที่ทำ

รอบ	วันที่	ขนาด	Features	วิธีแบ่ง	สถานะ
รุ่นแรก	24 มิ.ย. 69	300k	80	สุ่ม 80/20	เล็กใช้
แก้ class imbalance	6 ก.ค. 69	300k	80	สุ่ม	เหลือแต่รูป
300k 70/30	18 ก.ค. 69	300k	~77	70/30	เหลือแต่รูป
cicids2017_temporal_v1	25-27 ก.ค. 69	2.5M	77	ตามเวลา 70/30	ตัวปัจจุบัน

สามรอบแรกอยู่ใน `Result.zip` สองรอบกลางเก็บไว้เพียง confusion matrix ไม่มีไฟล์ metrics จึงนำมาเทียบตัวเลขไม่ได้ เก็บไว้เป็นประวัติเท่านั้น

รอบปัจจุบัน `cicids2017_temporal_v1`

ข้อมูล

	ค่า
แถวทั้งหมดหลัง clean	2,497,980
Train / Test	1,748,579 / 749,401
Features	77
Classes	9
Seed	42

การ clean ที่ใช้

ทำที่ละไฟล์แล้วค่อยรวม ตามลำดับนี้

1. ตัดช่องว่างหน้าชื่อคอลัมน์
2. บันทึก `_row_index` ก่อนทิ้งแถวใด ๆ เพื่อให้อ้างตำแหน่งแถวเดิมใน CSV ได้เสมอ
3. ปรับชื่อคอลัมน์อัตราให้ตรงกัน
4. ตัดคอลัมน์ซ้ำ `Fwd Header Length.1`
5. ตัดคอลัมน์ระบุตัวตน `Flow ID, Source IP, Destination IP, Source Port, Timestamp`
6. ตัด `Protocol` เพื่อไม่ให้กลายเป็นตัวบ่งบอกแหล่งที่มาของข้อมูล
7. ตัดแถว header ที่ปนมาเป็นข้อมูล
8. ยุบ sub-label เป็น 9 คลาสหลัก
9. แปลงเป็นตัวเลข, `inf` เป็น `NaN`, ทิ้งแถวที่มีค่าไม่สมบูรณ์
10. เก็บ feature เป็น `float32`
11. ตัดแถวซ้ำโดยเทียบ feature กับ label ไม่นับ metadata

เก็บ Destination Port ไว้ เพราะมีความหมายเชิงบริการ ต่างจาก source port

วิธีแบ่งข้อมูล

แบ่งตามลำดับเวลา 70/30 ภายในทุกกลุ่ม (ไฟล์ที่จับข้อมูล, คลาส) — 70% แรกของแต่ละคลาสใช้เทรน 30% ท้ายใช้ทดสอบ

ทำไมไม่กันทั้งไฟล์ออก เพราะใน CICIDS2017 แต่ละคลาสการโจมตีอยู่ในไฟล์เดียวกัน

ไฟล์	คลาสที่มี
Monday	BENIGN อย่างเดียว
Tuesday	Brute Force
Wednesday	DoS, Heartbleed
Thursday เช้า	Web Attack
Thursday บ่าย	Infiltration
Friday เช้า	Bot
Friday บ่าย DDoS	DDoS
Friday บ่าย PortScan	PortScan

กันไฟล์ได้ออกก็เท่ากับคลาสนั้นไม่มีข้อมูลเทรนเลย

ตัวนอกลำดับเวลา ไฟล์รุ่น MachineLearningCVE ไม่มีคอลัมน์ Timestamp จึงใช้ลำดับแถวในไฟล์ แทน และมีการตรวจสอบว่าลำดับนั้นตรงกับตารางเวลาการโจมตีที่ CIC ประกาศไว้จริงก่อนเทรนทุกครั้ง

ข้อควรระบุในรายงาน CICFlowMeter สร้าง record ตอน flow จบ ไม่ใช่ตอนเริ่ม ลำดับจึงเป็นลำดับ การจบ ซึ่งยังเป็นลำดับที่ถูกต้องสำหรับงาน IDS เพราะ feature vector ของ flow เกิดขึ้นได้ก็ต่อ เมื่อ flow จบแล้ว

การหุนที่แฟร์

ทุกโมเดลใช้วิธีเดียวกัน เป้าหมายเดียวกัน งบเท่ากัน (n_iter=20 บน 200,000 แถว) และ search space เท่ากันที่ 144 ชุด ทุกตัวจึงได้คั่น 13.9% เท่ากัน

ไม่หุน max_iter และ class_weight โดยตั้งใจ ตัวแรกจะทำให้เทียบโมเดลที่ยังไม่ converge กับ ที่ converge แล้ว ตัวหลังจะทำให้โมเดลหนึ่งเสี่ยงการจัดการ imbalance ที่ตกลงกันได้

ผลของรอบปัจจุบัน

โมเดล	macro-F1
lightgbm	0.8774
xgboost	0.8773
catboost	0.8873
random_forest	0.8404
stacking	0.6822
logistic_regression	0.5574
mlp	0.5369

คลาสที่มีปัญหา

Bot ติดเพดานเดียวกันทุกโมเดล — recall อยู่ระหว่าง 0.150 ถึง 0.161 ทั้งเจ็ดโมเดล

โมเดล	Bot recall	Bot F1
catboost	0.1607	0.2061
lightgbm	0.1538	0.2514
xgboost	0.1607	0.2413
random_forest	0.1504	0.0459
mlp	0.1607	0.0096
logistic_regression	0.1573	0.0023

เมื่อโมเดลเชิงเส้นกับ gradient boosting ที่ทุนแล้วได้ recall เท่ากัน เพดานนั้นอยู่ที่ข้อมูล ไม่ใช่ที่โมเดล เป็นอาการเดียวกับที่ฝั่ง 2018 เจอในคลาส SlowHTTPTest

สองคลาสที่ test flow น้อยเกินไป Heartbleed มี 4 แถว และ Infiltration มี 11 แถว รายงานจึง ระบุ macro-F1 สองแบบ คือครบ 9 คลาส และเฉพาะ 7 คลาสที่มี test flow อย่างน้อย 30 แถว

ไฟล์ในโฟลเดอร์นี้

ไฟล์	เนื้อหา
report/cicids2017_temporal_v1_report.md	รายงานฉบับเต็มของรอบปัจจุบัน
data/metrics.json	ผลรวมทุกโมเดล
data/<model>_per_class.csv	precision / recall / F1 รายคลาส

ผลดิบอยู่ที่ results/cicids2017_temporal_v1/ ข้อมูลที่ clean แล้วอยู่ที่ data/processed/cicids2017_clean.parquet และไฟล์ดิบอยู่ที่ data/raw/